When Does Reconfiguration Awareness Pay Off? An Exact-Oracle Study of Learned Link Sleeping in Software-Defined Networks

Adeel Ahmad[a,*], Toqeer Ali Syed[a], Ali Akarma[a,b]

[a] *AI Center, Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah, 42351, Saudi Arabia*
[b] *AI V&V Lab, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia*

Abstract

Learned controllers for energy-aware software-defined networks (SDNs) map each traffic matrix to a precomputed link-sleeping configuration and ignore the cost of changing configuration. We ask when that cost matters and how much of any gain needs learning. We formulate configuration over a finite library as a sequential problem that prices energy, reconfiguration, and wear, solve it exactly by dynamic programming, bound the library's loss with a per-interval integer program (at most 0.045%), and train LiRA, a history-conditioned imitation policy with a safety layer, as a strong learned controller. On the Abilene and GÉANT traces the optimal sequence is a single configuration, and snapshot learning, hysteresis, and LiRA reach the oracle cost to within 0.014%. On a semi-synthetic Nobel-Germany series, a gradient-boosted snapshot classifier flaps (11.7 changes per day) and costs 6.5% more than the oracle. Holding a configuration removes 97.7% of the changes at no energy loss, but tuned hysteresis, a rolling-horizon optimizer, and a static configuration with LiRA's fallback do the same: the static rule equals LiRA in 11 of 12 settings, and no difference survives a correction over all settings. The conclusions hold without the wear price and across noise variants, where the snapshot excess ranges from 0.0% to 32.5% with the noise's time correlation and scale. With the previous interval's traffic, costs barely change but the utilization bound is violated in up to 0.19% of the

*Corresponding author.
Email address: 443057803@stu.iu.edu.sa (Adeel Ahmad)

intervals. Reconfiguration awareness pays off only against flapping classifiers, and a rule that holds its configuration captures that benefit.

Keywords: Software-defined networking, energy efficiency, link sleeping, imitation learning, dynamic programming, reconfiguration cost, device lifetime

---

1. Introduction

Backbone and data-center links are provisioned for peak demand, yet their average utilization is far lower, so a large share of the power drawn by line cards and transceivers is spent carrying little or no traffic. Putting idle links to sleep was proposed more than two decades ago [1] and has since grown into a broad research area [2, 3]. Software-defined networking (SDN) makes the idea practical, because a logically centralized controller can observe the traffic matrix and switch link power states and forwarding rules network-wide [4, 5]. The underlying optimization is a mixed-integer problem that is NP-hard in general [6], and an exact solver needs up to 11.4 s per traffic matrix on the topologies we study (Section 7.6), a time that grows with the network and limits how often a controller can decide.

Recent work closes this speed gap with machine learning. A model is trained offline to map each incoming traffic matrix to one of a set of precomputed, energy-efficient configurations, which replaces the integer program at run time [7, 8]. The same pattern of replacing a solver with a learned mapping appears in traffic engineering [9, 10, 11, 12] and in combinatorial optimization more broadly [13].

A snapshot decision treats every interval as the first. The configuration chosen for the current traffic matrix ignores which configuration is installed, although moving between configurations has costs. Links must be woken or put to sleep, flow tables must be rewritten, and updates that are not carefully ordered can cause transient loops or loss [14, 15]. Each power-state transition also adds a thermal cycle that can offset the lifetime benefit of sleeping [16, 17]. The hypothesis is that snapshot classifiers flap between neighboring configurations when demand hovers near a decision boundary, and that a sequence-aware method should avoid this at little cost in energy.

This paper tests that hypothesis rather than assuming it. Its question is not whether a new controller beats old ones, but when the sequence matters at all, and how much of any gain needs learning. To answer it we need a

reference that knows the best achievable sequence, a controller that can learn it, and baselines that hold a configuration without learning. We make four contributions.

- *Formulation and exact oracle.* We formulate energy-aware configuration as a sequential problem that jointly prices energy, reconfiguration, and wear. Over a finite library, the problem is a shortest path on a trellis, which dynamic programming solves exactly in $O(TK^2)$ time. We validate the solver against an independent mixed-integer program to 1×10^{-14} and prove that snapshot decisions can have regret that grows linearly with the horizon (Sections 3 and 4). A per-interval integer program bounds what the library restriction can cost (Section 7.7).

- *A representative sequence-aware learner.* LiRA (Lifetime- and Reconfiguration-Aware learning) imitates the oracle with cost-sensitive weights and dataset aggregation (DAgger) [18], conditions on the installed configuration and its dwell time, and wraps the policy in a feasibility safety layer (Sections 4 and 5). It serves as the strongest learned controller we could build from the oracle, so that a null result cannot be blamed on a weak learner. Drift detectors are described for completeness, and Section 7.10 reports that our drift study does not validate them.

- *An empirical answer to the question.* We evaluate on Abilene, GÉANT, and a semi-synthetic Nobel-Germany series under four load and power scenarios, against snapshot learning, hysteresis, minimum-dwell, rolling-horizon, static-configuration, integer-programming, genetic-algorithm, and greedy baselines. We separate the effect of the wear model by repeating the study without a wear price, and the effect of the synthetic noise by repeating it with temporally correlated and weaker noise. Ablations, decisions on one-interval-old traffic, a drift study, and rolling-origin folds complete the study (Sections 6 and 7).

- *Emulation of configuration changes.* A Mininet and Open vSwitch emulation replays the most active day of the Nobel-Germany series and measures rule-installation time, the duration of a configuration change, probe loss, and delay for each method (Section 7.13).

Our answer is a qualified one. On the two real traces, the energy-optimal configuration changes rarely: the exact oracle uses one configuration for the

whole test period, snapshot classifiers change configuration at most 0.019 times per day, and LiRA, hysteresis, and the snapshot classifier reach the same energy savings (24.0% on Abilene and 29.8% on GÉANT) at costs within 0.014% of each other. LiRA therefore offers no measurable advantage there. On the Nobel-Germany series, whose demands are noisy, the snapshot classifier flaps (11.7 changes per day), and 99% of its changes save less than 0.1% of the interval energy. LiRA removes 97.7% of the changes, but in this scenario its trained policy is a constant classifier, and three rules without learning remove them as well: hysteresis tuned on validation data, a rolling-horizon optimizer with a persistence forecast, and a static configuration with LiRA's fallback. LiRA's learned component lowers the cost below that of the static rule in one scenario only, combined high load and high dynamic power. There its advantage over hysteresis is 0.09% and not significant, and the rolling-horizon optimizer is cheaper still. The same holds without the wear price and on three further noise variants of the Nobel-Germany series, in which the excess cost of the snapshot classifier ranges from 0.0% to 32.5%. The benefit of reconfiguration awareness therefore depends on the traffic and on the classifier, and in our data it comes from holding a configuration rather than from learning when to switch.

The remainder of the paper reviews related work (Section 2), formulates the problem (Section 3), presents LiRA (Sections 4 and 5), describes the setup (Section 6), reports results (Section 7), and discusses implications, limitations, and conclusions (Sections 8–10).

2. Related Work

2.1. Energy-Aware Networking and Sleep Modes

Surveys of green networking classify techniques into adaptive link rate, sleeping, and energy-aware routing [2, 3]. Chiaraviglio *et al.* formulated ISP energy minimization as an integer program and proposed heuristics that switch off nodes and links subject to capacity and connectivity constraints [6]. GreenTE maximizes the number of links that can sleep under utilization and delay constraints [19], ElasticTree applies the same principle to data-center networks [20], and EATe spreads traffic online so that lightly loaded links can sleep [21]. Power models for such studies rely on measured device profiles [22, 23, 24]. In SDN, energy-aware routing has been formulated as integer programs with heuristics for carrier-grade Ethernet [25], and a survey of SDN-specific methods is given in [26].

2.2. Learning for Energy-Efficient Networking

MER-SDN uses machine learning to predict the parameters of an energy-aware routing heuristic [27]. Reinforcement learning has been applied in an SDN controller to trade packet delay against energy [28] and to compute energy-saving routes in data-center networks [29]. In cellular networks, deep reinforcement learning combined with traffic forecasting controls base-station sleep modes [30], a problem that shares the on-off structure of link sleeping. Deep reinforcement learning has also been used to put switches and links of data-center networks to sleep [31] and to route traffic in SDN [32], and energy-aware traffic engineering has been adapted to hybrid SDN deployments [33]. A recent study of two European ISP networks shows that more than a third of the links could sleep without congestion [34], which confirms that the opportunity measured on research backbones exists in operational networks. The closest works to ours replace the optimizer with a learned mapping from traffic matrices to precomputed configurations and report large speedups [7, 8]. We reimplement this snapshot approach as baseline B4 with our own library, features, and classifier, because we did not have access to the original code. The comparison therefore concerns the snapshot principle and not the exact published systems.

2.3. Learning to Optimize and Learned Traffic Engineering

Machine learning for combinatorial optimization treats problem instances as data and learns to imitate expert decisions or to replace parts of a solver [13]. In traffic engineering, learning-accelerated and decomposition-based solvers [9, 10, 11, 35], learned routing [36], deep reinforcement learning with graph neural networks [12, 37], and neural models of network performance [38] have shown that learned components can approach solver quality at much lower latency. Most of these methods do not price the change from one configuration to the next. FlexDATE is an exception: it limits the disturbance of rerouting in reinforcement-learning traffic engineering [39], but it does not put links to sleep. LiRA instead imitates an oracle that is exact for the sequential objective, which the finite library makes tractable offline.

2.4. Reconfiguration Cost, Device Lifetime, and Switching Costs

Consistent-update abstractions [14] and update scheduling [15] show that changing forwarding state is neither free nor instantaneous. Bonetto *et al.* included reconfiguration costs in multi-period power-aware logical topology

design [40]. Chiaraviglio *et al.* modeled how sleep modes affect device lifetime through temperature and power-state transitions [16, 17], and lifetime-aware provisioning has been studied for optical backbones [41]. Online problems with switching costs are studied as metrical task systems [42] and smoothed online convex optimization [43], with dynamic right-sizing of data centers as a well-known application [44]. Our setting is a metrical task system with a finite state space whose offline optimum is computable, which makes imitation learning natural [18]. Learning-augmented online algorithms combine a learned predictor with a robust online rule, with guarantees for caching [45], general metrical task systems [46, 47], and smoothed online optimization [48]. They show that following a predictor blindly can be costly, which is the flapping we measure, and that a rule which holds its state is the natural robust fallback. Our hysteresis and static baselines are simple instances of such rules. Concept-drift adaptation [49, 50] with detectors such as ADWIN [51] and cumulative-sum tests [52] provides the tools for deciding when to retrain, and drift is known to degrade learned models in network security [53].

3. Problem Formulation

Table 1 summarizes the notation.

3.1. Configuration Model

The network is a graph $G = (V, E)$ in which each undirected link e has capacity κ_e in each direction and consists of two ports that sleep and wake together. Time is divided into control intervals of length Δ, and D_t is the traffic matrix of interval t. A configuration $c = (x, r)$ consists of a link-state vector $x \in \{0,1\}^{|E|}$, with $x_e = 1$ for an active link, and a routing r that splits each origin–destination demand over up to k precomputed paths that use only active links. A library $\mathcal{C} = \{c_1, \ldots, c_K\}$ of such configurations is built offline (Section 4.1). The utilization $u_e(c, D)$ of link e in a direction is the offered load divided by κ_e, and c is feasible for D if its maximum link utilization (MLU) satisfies

$$\mathrm{MLU}(c, D) = \max_{e:\, x_e = 1,\ \mathrm{dir}} u_e(c, D) \le \theta \tag{1}$$

for a utilization threshold $\theta < 1$.

Table 1: Main Notation

Symbol	Meaning
$G = (V, E)$	Network graph; E is the set of undirected links
κ_e	Capacity of link e in each direction
Δ, n_d	Control interval; number of intervals per day
D_t, $\hat{D}_t$	Traffic matrix of interval t; its estimate
$c = (x, r)$	Configuration: link states $x \in \{0,1\}^{\lvert E\rvert}$, routing r
$\mathcal{C}$, K	Configuration library and its size
u_e, θ	Utilization of link e; utilization threshold
$E(c, D)$	Energy of c under traffic D in one interval
$R(c', c)$	Reconfiguration cost of moving from c' to c
AF_e, α, χ	Acceleration factor; sleep factor; cycling weight
$W(c', c)$	Per-step wear cost
λ_R, λ_W	Prices of reconfiguration and wear
J, V_t, Q_t	Total cost; cost-to-go; action value

3.2. Energy Model

Chassis stay powered, so the controllable power is that of link ports. Following load-dependent port profiles [23, 22, 24], the energy of c in interval t is

$$E(c, D_t) = \Delta \sum_{e \in E} \Big[x_e \big(2P^{\mathrm{idle}} + P^{\mathrm{dyn}} (u_e^{\rightarrow} + u_e^{\leftarrow}) \big) + (1 - x_e)\, 2P^{\mathrm{sl}} \Big], \tag{2}$$

where P^{idle}, P^{dyn}, and P^{sl} are the idle, full-load dynamic, and sleep power of one port, and $u_e^{\rightarrow}, u_e^{\leftarrow}$ are the directional utilizations. We report energy in units of one interval of the idle power of all links, so that the energy of the all-on configuration is close to one per interval.

3.3. Reconfiguration Cost

Moving from c' to c toggles $\|x - x'\|_1$ links, changes $\Delta F(c', c)$ forwarding entries (the number of switch and origin–destination pairs whose next hops differ), and moves a share $B(c', c) \in [0, 1]$ of the traffic to different paths,

measured on the mean training traffic matrix so that R is a fixed table over library pairs. The reconfiguration cost is

$$R(c', c) = \tfrac{1}{3}\Big[\frac{\|x - x'\|_1}{\rho_1} + \frac{\Delta F(c', c)}{\rho_2} + \frac{B(c', c)}{\rho_3}\Big], \tag{3}$$

where ρ_1, ρ_2, ρ_3 are the 95th percentiles of the three terms over distinct library pairs, so that a typical large change costs about one unit.

3.4. Wear Model

We adopt the acceleration-factor view of Chiaraviglio *et al.* [16]: relative to an always-on device, sleeping lowers the failure rate, whereas power-state transitions raise it through thermal cycling. Let τ_e be the fraction of time link e sleeps and f_e its number of transitions per day. We use the linear form

$$\mathrm{AF}_e = (1 - \tau_e) + \alpha\,\tau_e + \chi f_e = 1 - (1 - \alpha)\tau_e + \chi f_e, \tag{4}$$

where $\alpha < 1$ is the relative failure rate while asleep and χ weighs one transition per day. A value $\mathrm{AF}_e > 1$ means that the link ages faster than an always-on link. Over a horizon of T intervals, $\sum_t W(c_{t-1}, c_t) = \frac{T}{n_d} \sum_e (\mathrm{AF}_e - 1)$ for the additive per-step term

$$W(c', c) = \sum_{e \in E} \Big[\chi\,|x_e - x'_e| - \frac{1 - \alpha}{n_d}(1 - x_e)\Big]. \tag{5}$$

This decomposition is exact and makes wear additive over time. The parameters α and χ are assumed, representative values (Section 6.5), and we study the sensitivity of the results to them.

3.5. Sequential Objective

Given an initial configuration c_0, the sequential problem selects $c_1, \ldots, c_T \in \mathcal{C}$ that minimize

$$J(c_{1:T}) = \sum_{t=1}^{T} \Big[E(c_t, D_t) + \lambda_R R(c_{t-1}, c_t) + \lambda_W W(c_{t-1}, c_t)\Big] \tag{6}$$

subject to (1) in every interval, where $\lambda_R, \lambda_W \geq 0$ set the price of reconfiguration and wear relative to energy. Snapshot methods instead use the per-interval rule

$$c_t^{\mathrm{snap}} = \arg \min_{c \in \mathcal{C}:\, \mathrm{MLU}(c, D_t) \leq \theta} E(c, D_t), \tag{7}$$

which ignores the installed configuration. Online, the controller needs an estimate of D_t from flow counters or link loads [54], and Section 6.3 states what our evaluation assumes about it.

3.6. Cost of Ignoring the Sequence

For two configurations c_a and c_b, let $s_{ab} = \lambda_R R(c_a, c_b) + \lambda_W \chi \|x_a - x_b\|_1$ be the non-energy cost of switching between them.

Proposition 1. *Let $c_a, c_b \in \mathcal{C}$ be feasible in every interval and have the same number of sleeping links, and let the traffic alternate so that $E(c_a, D_t) - E(c_b, D_t) = (-1)^t \epsilon$ with $\epsilon > 0$, while every other library member is dominated in every interval. Then the snapshot rule (7) incurs a regret of at least $(T-1)s_{ab} - \lceil T/2 \rceil \epsilon$ with respect to the optimum of (6), which grows linearly in T whenever $s_{ab} > \epsilon/2$.*

Proof. The snapshot rule selects c_b in even and c_a in odd intervals, so it pays the lower energy in each interval and switches $T-1$ times. The sleep terms of (5) are identical for c_a and c_b, so the two sequences differ only in energy and in switching cost. Staying at c_a throughout pays at most $\lceil T/2 \rceil \epsilon$ more energy and never switches. Its cost is an upper bound on the optimum, which proves the claim. □

The proposition shows that flapping can be costly in principle. Whether it is costly in practice depends on whether real traffic produces such near-ties, which is the empirical question of Section 7.

4. LiRA: Framework and Analysis

Fig. 1 shows the architecture. An offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. An online plane runs once per interval: it estimates the traffic matrix, builds features, queries the policy, checks feasibility, and installs only the difference between the current and the new configuration. Drift detectors connect the two planes.

4.1. Configuration Library

The library must be small enough for fast decisions and rich enough to contain an efficient feasible configuration for almost every traffic matrix. We cluster training traffic matrices with k-means (64 clusters) and take the element-wise 95th-percentile matrix of each cluster as its representative. For each representative we solve the path-based energy-minimization integer program with $k = 4$ shortest paths, link-pair sleeping, connectivity, and the

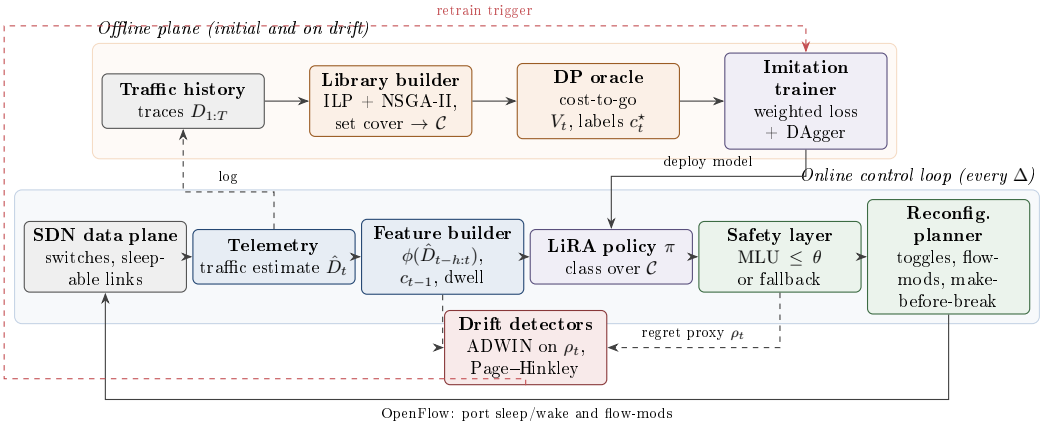


Figure 1: Architecture of LiRA. The offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. The online loop estimates the traffic matrix, selects a configuration conditioned on the installed one, enforces the utilization bound, and installs only the configuration difference. Drift detectors trigger retraining.

bound (1), and we run NSGA-II [55] on port count and maximum utilization to add Pareto-diverse link-state vectors. For Abilene, whose link-state space is small, we also enumerate all connected link states. Duplicate vectors are merged, the routing of each vector is re-optimized by a linear program that minimizes maximum utilization, and further integer programs are solved for uncovered training and validation matrices until at least 99.5% are covered. The all-on configuration is always included. The library is pruned to K configurations by greedy energy coverage on every sixth training interval. Section 7.7 bounds what this restriction costs.

4.2. Exact Sequence-Optimal Oracle

Because all terms of (6) depend only on (c_{t-1}, c_t, D_t), the problem is a shortest path through a trellis whose stages are intervals and whose states are library members (Fig. 2). Infeasible states receive infinite cost. The backward recursion

$$\begin{aligned} V_{t-1}(c') &= \min_{c \in \mathcal{C}} Q_t(c', c), \\ Q_t(c', c) &= E(c, D_t) + \lambda_R R(c', c) + \lambda_W W(c', c) + V_t(c) \end{aligned} \tag{8}$$

computes the cost-to-go $V_t(c)$ of being in c at the end of interval t, with $V_T \equiv 0$. The oracle trajectory follows $c_t^\star = \arg\min_c Q_t(c_{t-1}^\star, c)$. The table

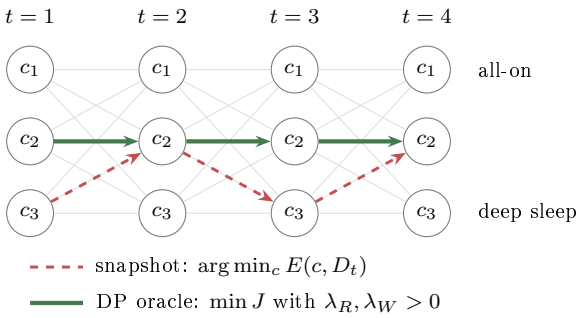


Figure 2: Configuration trellis (schematic). Nodes are library configurations per interval, and edges carry reconfiguration and wear costs. Snapshot selection flaps between c_2 and c_3 when traffic hovers near a decision boundary, whereas the oracle keeps c_2 when the extra energy is smaller than the switching cost.

$Q_t(c', \cdot)$ also gives the optimal action for *any* installed configuration c', which we use to relabel states visited by the learner.

Proposition 2. *For a fixed library $\mathcal{C}$ and known $D_{1:T}$, recursion* (8) *returns a feasible sequence minimizing* (6) *whenever one exists, in $O(TK^2)$ time and $O(TK)$ memory.*

Proof. By (5), the wear contribution is a sum of terms that each depend on one pair of consecutive configurations, and (2) and (3) depend on (c_{t-1}, c_t, D_t) only. Hence (6) is the length of a path through a layered graph with K nodes per layer, and edges into infeasible nodes have infinite weight. Bellman's principle of optimality yields (8), and each layer requires K^2 evaluations of precomputed quantities. □

4.3. History-Conditioned Policy

The policy maps the input s_t to a library index. The input contains the principal components (95% variance, fitted on training data) of the log-scaled origin–destination vector, the total volume, the shares of the ten largest demands, time of day and day of week, the changes of the principal components and the volume over the last 1, 3, and 6 intervals, the one-hot encoding of the installed configuration c_{t-1}, and its dwell time ℓ_{t-1} in intervals, capped at one day. The default model is a gradient-boosted tree classifier (XGBoost [56]), and Section 7.3 compares LightGBM [57], random forests [58], and a multilayer perceptron.

Cost-sensitive imitation. Plain classification treats all mistakes as equal, although choosing an action that is almost as cheap as the optimum is harmless. For each training state (t, c') we compute the oracle action $a^\star = \arg\min_c Q_t(c', c)$ and the cost gap $g_t(c') = \min_{c \neq a^\star} Q_t(c', c) - Q_t(c', a^\star)$ to the second-best action. The classifier is trained with the sample weight $w = \mathrm{clip}(g/\bar{g}, 0.2, 5)$, where $\bar{g}$ is the mean gap, so that decisions with a large cost difference matter more. This weighted cross-entropy loss is a practical surrogate for a cost-sensitive classifier.

DAgger. A policy trained only on oracle trajectories never sees the states that its own mistakes create. We apply DAgger [18] with three rounds: the current policy is rolled out on the training period, with the expert action used with probability $\beta_n = 0.5 \cdot 0.5^{n-1}$ in round n, the visited pairs (t, c_{t-1}) are relabeled from the stored Q tables without solving any new problem, and the policy is retrained on the aggregated data. Among the rounds, we keep the policy with the lowest sequence cost on the validation period. In 56 of our 60 main runs this is the policy of round zero, trained on oracle trajectories alone.

Remark. Because the oracle's table Q_t gives the optimal action from every installed configuration, the DAgger analysis for task costs [18] applies with a per-deviation loss of at most $u = \max_{t,c',c}[Q_t(c', c) - \min_{\tilde{c}} Q_t(c', \tilde{c})]$, so the cost gap grows linearly in the learner's disagreement with the oracle rather than quadratically in T. Our weighted cross-entropy only approximates the cost-sensitive loss of that analysis, so this motivates the design and is not a guarantee for the implemented learner.

4.4. Safety Layer

At run time, the configuration chosen by the policy is checked against (1) using the traffic estimate. If it is infeasible, the controller falls back to

$$c_t = \operatorname*{arg\,min}_{c \in \mathcal{C}:\, \mathrm{MLU}(c, \hat{D}_t) \leq \theta} \Big[E(c, \hat{D}_t) + \lambda_R R(c_{t-1}, c) + \lambda_W W(c_{t-1}, c) \Big], \tag{9}$$

which is evaluated for all K configurations. If no configuration is feasible, the all-on configuration is used and the interval is counted as a violation.

Proposition 3. *If the all-on configuration satisfies* $\mathrm{MLU} \leq \theta$ *for* $\hat{D}_t$*, the safety layer returns a configuration that satisfies* (1) *for* $\hat{D}_t$*.*

Proof. The all-on configuration belongs to $\mathcal{C}$ by construction, so the feasible set in (9) is non-empty whenever the policy's choice is infeasible. □

The guarantee holds for the estimate $\hat{D}_t$, not for the traffic that then arrives. Section 6.3 states which estimate our evaluation uses, and Section 7.11 measures the violations when the estimate is one interval old.

4.5. Drift Detection and Retraining

The oracle cannot be computed online, but a myopic regret proxy can. After each decision, the controller computes

$$\begin{aligned}\rho_t = & E(c_t, D_t) - \min_{c \in \mathcal{C}:\, \mathrm{MLU}(c, D_t) \le \theta} E(c, D_t) \\ & + \mathbb{1}[\mathrm{MLU}(c_t, D_t) > \theta],\end{aligned} \tag{10}$$

the energy lost against the best feasible configuration plus a violation indicator. ADWIN [51] monitors ρ_t, and a Page–Hinkley test [52] monitors the deseasonalized log of the total traffic volume, that is, the log volume minus its training mean for the same time of week. When a detector fires, the policy is retrained on the last seven days with oracle labels computed on that window. Because the window just after a change still contains mostly old data, retraining is repeated one and three days later. A library refresh, which adds integer-program solutions for eight clusters of the recent window, is available as an option.

5. Algorithms

Algorithm 1 summarizes offline training and Algorithm 2 the online loop. In the emulation (Section 7.13), reconfigurations follow a make-before-break order: links that the new configuration needs are woken first, new rules are installed, and only then are unused links put to sleep. The flow-level simulation charges each change through R and W and does not model the transient.

6. Experimental Setup

6.1. Datasets and Scenarios

Table 2 lists the data. Abilene (Zhang *et al.* [54]) and GÉANT (Uhlig *et al.* [59]) topologies and traffic matrices come from SNDlib [60]. SNDlib

Algorithm 1 LiRA Offline Training

Input: training traces $D_{1:T}$, validation traces, prices λ_R, λ_W, rounds N
Output: library $\mathcal{C}$, policy π

$\mathcal{C} \leftarrow$ BUILDLIBRARY$(D_{1:T})$ ▷ ILP, NSGA-II, set cover
precompute $E(c, D_t)$, feasibility, and R, W for all pairs
compute $Q_t(c', c)$ and $V_t(c)$ by recursion (8)
$\mathcal{D} \leftarrow \{(s_t, c_t^\star)\}$ on the oracle path (s_t contains $c_{t-1}^\star$), weights w
train π on $\mathcal{D}$
for $n = 1$ to N **do**
    roll out π with the safety layer and expert mixing β_n; record (t, c_{t-1})
    relabel the visited pairs from $Q_t(c_{t-1}, \cdot)$ and add them to $\mathcal{D}$
    retrain π on $\mathcal{D}$
end for
return $\mathcal{C}$ and the policy with the lowest validation cost

offers only one day of five-minute matrices for Nobel-Germany, while day-of-week features and a chronological split need several weeks. We therefore generate a 30-day semi-synthetic series: a gravity model fitted to the SNDlib day is modulated by the weekly profile of Abilene and multiplied by independent log-normal noise per origin–destination pair and interval, whose scale ($\sigma = 1.23$) is fitted to the residuals of Abilene. This noise has no temporal correlation and is therefore harsher than the noise of a real trace.

We removed unfilled gaps and all-zero measurement outages, interpolated gaps of up to three intervals, removed eleven GÉANT intervals whose total volume exceeds ten times its one-day rolling median, and split each series chronologically into 60% training, 15% validation, and 25% test. Sequences never cross an unfilled gap. Traffic is scaled so that the 99.9th percentile of the all-on maximum link utilization on the training split is 0.5, which makes the scenario "base".

We add three scenarios to probe regimes in which sequence decisions could matter more: *high load* (demands $\times 1.6$, a training peak of 0.8), *high dynamic power* (the load-proportional port power tripled to 15 W), and *stress* (both). The library is rebuilt for each scenario.

6.2. Baselines

B0 keeps all links on and routes each demand on its shortest path. B4 is our reimplementation of the snapshot-learning principle [7, 8]: an XGBoost classifier that maps the current traffic features to the energy-optimal feasible library member. If its choice violates θ, B4 falls back to the feasible config-

Algorithm 2 LiRA Online Control Loop

Input: $\mathcal{C}$, π, threshold θ, initial c_0

for each interval $t = 1, 2, \ldots$ **do**
  build s_t from the traffic estimate, c_{t-1}, and its dwell time
  $c \leftarrow \pi(s_t)$
  if $\mathrm{MLU}(c, \hat{D}_t) > \theta$ **then**
    $c \leftarrow$ fallback rule (9)
  end if
  if $c \neq c_{t-1}$ **then**
    wake links in $x \setminus x_{t-1}$; install new rules
    sleep links in $x_{t-1} \setminus x$
  end if
  $c_t \leftarrow c$; observe D_t; update detectors with ρ_t from (10)
  if a detector fires **then**
    schedule retraining on the last seven days (Section 4.5)
  end if
end for

Table 2: Topologies and Traffic Data

Topology	$\lvert V\rvert$	$\lvert E\rvert$	Δ	Intervals	Span	Traffic
Abilene	12	15	5 min	48 108	2004-03-01 to 2004-09-10	real (SNDlib)
GÉANT	22	36	15 min	10 784	2005-05-04 to 2005-08-31	real (SNDlib)
Nobel-Germany	17	26	5 min	8 640	2004-03-01 to 2004-03-30	semi-synthetic

uration with the lowest energy. B6 applies hysteresis to B4, switching only if the energy gain of the proposed configuration exceeds a threshold, and B7 applies a minimum dwell time. Both thresholds are tuned on the validation split by sequence cost J. B2 is a rolling-horizon controller that solves the sequential problem exactly over the library for the next 12 intervals with a persistence forecast and applies the first action. It needs no learning and is restricted to the library. The static rule holds the configuration that the oracle uses most often on the training period and applies LiRA's fallback (9) whenever that configuration is infeasible. It is LiRA with its classifier replaced by a constant. B1 solves the energy integer program for each traffic matrix with HiGHS [61], B3 runs NSGA-II per traffic matrix, and B5 is an ElasticTree-style greedy heuristic [20]. B1, B3, and B5 are evaluated on a subsample of 60 test intervals (15 for B3 on Abilene) because of their run time. We also report the offline oracle $\mathrm{O}^\star$, which uses the future and is a lower bound for every controller restricted to the library. Section 7.7

bounds controllers that are not. A deep reinforcement learning baseline is not included.

6.3. Information Available to the Controller

Unless stated otherwise, every online method chooses the configuration of interval t with the realized matrix D_t: its features are computed from D_t, and its safety check uses $\mathrm{MLU}(c, D_t)$. This idealizes the estimate $\hat{D}_t$ of Algorithm 2, and it gives the safety layer the information that the oracle has. Section 7.11 repeats the comparison when every online method decides with D_{t-1}, the last completed measurement, with and without a safety margin of 0.05 on θ.

6.4. Metrics and Prices

Energy savings S are measured relative to B0. We report configuration changes and link power-state transitions per day, flow-modification messages, the share of intervals with a violation of θ, the sequence cost J of (6), the mean and maximum acceleration factor, and the decision latency measured with batch size one. The retention $X = S_{\mathrm{LiRA}}/S_{\mathrm{B4}}$ and the reductions $Y = 1 - N^{\mathrm{ch}}_{\mathrm{LiRA}}/N^{\mathrm{ch}}_{\mathrm{B4}}$ and $Z = 1 - N^{\mathrm{tr}}_{\mathrm{LiRA}}/N^{\mathrm{tr}}_{\mathrm{B4}}$ are defined only when B4 changes configuration. The reported cost J is (6) without the constraint: an interval that violates θ adds no penalty and is counted in the violation share instead, and the first test interval and the first interval after a measurement gap carry no switching cost. J also includes the modeled wear credit of sleeping links, so it is not an energy figure: absolute values of J are lower than the energy of the always-on network, and only differences between methods are meaningful. With the default prices a sleeping link earns a wear credit of 0.040 per interval, while sleeping it saves 0.060, 0.035, and 0.025 of the normalized energy on Abilene, Nobel-Germany, and GÉANT, so the wear model rewards sleeping about as much as the energy model does.

The prices are set from operating considerations. One typical large change costs 0.2% of the daily energy of the all-on network, so $\lambda_R = 0.002\, n_d$, and one link transition costs 0.08% of it, so $\lambda_W = 0.0008\, n_d/\chi$, where n_d is the number of intervals per day. For stochastic components (classifier seeds and DAgger rollouts) we use five seeds and report means with 95% confidence intervals. Where a policy is constant, all seeds give the same result and the confidence interval has zero width. We compare the per-day sequence costs of LiRA with those of B4, B6, and B7 with the Wilcoxon signed-rank test and correct the three comparisons per setting with the Holm method.

Table 3: Model and Training Parameters

Parameter	Value
Threshold θ; paths k; library K	0.8; 4; 32 (default)
Port power $P^{\text{idle}}/P^{\text{dyn}}/P^{\text{sl}}$	30/5/3 W
Wear α; χ	0.5; 0.01 per transition per day
Prices λ_R; λ_W	$0.002\,n_d$; $0.0008\,n_d/\chi$
Interval Δ; n_d	5 min, 288 (GÉANT: 15 min, 96)
XGBoost	depth 6, rate 0.1, 150 rounds max, early stop 15
Features	PCA 95% variance, lags 1/3/6
DAgger rounds; β_1; weight clip	3; 0.5; $[0.2, 5]$
ADWIN confidence; Page–Hinkley tolerance, forgetting	0.002; 0.005, 0.9999
Retraining window; schedule	7 days; at detection, +1 and +3 days
Seeds; folds	5; 4 rolling origins

Because twelve settings are tested, we also apply the Holm correction to all 36 comparisons at once and report the result. Confidence intervals that round to zero, which occur when every seed gives the same sequence, are not printed.

6.5. Implementation and Parameters

The flow-level simulator is written in Python with NumPy and sparse path–link matrices. Integer and linear programs are solved with HiGHS (highspy 1.15.1). Experiments ran on an Intel Core i9-13900H laptop (14 cores, 20 threads, 48 GB RAM, no GPU) under Windows, and the emulation ran in Ubuntu on WSL2 with Mininet 2.3.0 and Open vSwitch 3.7.1. Table 3 lists the parameters. The port power values are representative assumptions within the ranges reported in [24, 22], not measurements of a specific device, and the wear parameters are assumed values for the model of [16]. We therefore study the sensitivity to both (Section 7.5).

7. Results

Two facts about the trained LiRA policies frame all results. In 30 of the 60 main runs (all Abilene runs and the Nobel-Germany base and high-dynamic-power scenarios) the trained policy is a constant classifier, because the oracle uses a single configuration on the training period. LiRA then holds the configuration that was optimal on the training period and switches only through its fallback, and its ablations are trivial. In 56 of the 60 runs the policy selected on validation data is that of round zero, before any DAgger data. The static rule of Section 6 makes the consequence measurable: it reaches the same cost as LiRA in 11 of the 12 settings of Table 5.

7.1. Do Real Traces Need Sequence Awareness?

Table 4 gives the main comparison in the base scenario. On Abilene and GÉANT, B4, B6, B7, LiRA, and the oracle reach the same energy savings (24.0% and 29.8%), make at most 0.019 configuration changes per day, and have sequence costs within 0.014% of each other. The rolling-horizon controller B2, which relies on a persistence forecast, is marginally worse on Abilene. The oracle itself uses a single configuration for the whole test period (41.8 and 28.1 days). B4 never changes configuration on GÉANT, so the reductions Y and Z are undefined there, and on Abilene they would rest on less than one change on average over the test period. LiRA is no worse and no better. The per-day cost differences between LiRA and B4, B6, and B7 are not significant on these traces (Table 5).

The Nobel-Germany series behaves differently. B4 changes configuration 11.7 times per day and makes 23.4 link transitions per day, whereas the exact snapshot rule (7) would change 63.7 times per day. LiRA makes 0.27 changes per day, a reduction of $Y = 97.7\%$ and $Z = 97.7\%$ at an energy retention of $X = 100.0\%$, and the mean acceleration factor drops from 0.855 for B4 to 0.846. The reduction does not come from learning. The policy is constant in this scenario and holds one configuration in 2 159 of the 2 160 test intervals, and the static rule gives the same result. Hysteresis (B6) and the rolling-horizon B2 both make 0.13 changes per day at a cost of 875.3, slightly below LiRA's 875.9, so LiRA is not better than either. The paired test gives an adjusted $p = 0.88$ against B6, with a mean daily cost difference (LiRA minus B6) of 0.07 and LiRA cheaper on 38% of the days, and $p = 0.023$ against B4. These reductions are relative to a gradient-boosted B4 that flaps heavily. A random-forest B4 changes configuration only 0.5 times per

Table 4: Main Comparison in the Base Scenario (Test Split). Means with 95% Confidence Intervals over Five Seeds for B4, B6, B7, and LiRA; Single Values Otherwise. Latency Is the Median Decision Time

Method	S (%)	Changes/d	Transitions/d	MLU viol. (%)	J	$\overline{\mathrm{AF}}$	Latency (ms)
Abilene							
B0 always-on	0.00	0.00	0.00	0.00	12037.5	1.000	–
B4 snapshot ML	23.98	0.02±0.05	0.08±0.21	0.00	7227.7±2.8	0.867	0.989
B6 B4+hysteresis	23.98	0.00	0.00	0.00	7226.7	0.867	0.989
B7 B4+min-dwell	23.98	0.02±0.05	0.08±0.21	0.00	7227.7±2.8	0.867	0.989
B2 rolling horizon	23.95	0.00	0.00	0.00	7230.4	0.867	0.433
LiRA	23.98	0.00	0.00	0.00	7226.7	0.867	0.008
Oracle O*	23.98	0.00	0.00	0.00	7226.7	0.867	–
GÉANT							
B0 always-on	0.00	0.00	0.00	0.00	2705.2	1.000	–
B4 snapshot ML	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B6 B4+hysteresis	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B7 B4+min-dwell	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B2 rolling horizon	29.85	0.00	0.00	0.00	603.7	0.833	0.432
LiRA	29.85	0.00	0.00	0.00	603.7	0.833	1.571
Oracle O*	29.85	0.00	0.00	0.00	603.7	0.833	–
Nobel-Germany							
B0 always-on	0.00	0.00	0.00	0.05	2163.7	1.000	–
B4 snapshot ML	27.63	11.68±0.89	23.36±1.78	0.00	931.7±4.3	0.855±0.001	0.949
B6 B4+hysteresis	27.63	0.13	0.27	0.00	875.3	0.846	0.949
B7 B4+min-dwell	27.63	2.29±0.18	4.59±0.36	0.00	885.8±0.9	0.848	0.949
B2 rolling horizon	27.63	0.13	0.27	0.00	875.3	0.846	0.423
LiRA	27.63	0.27	0.53	0.00	875.9	0.846	0.008
Oracle O*	27.63	0.00	0.00	0.00	874.7	0.846	–

day (Section 7.3), so the size of Y and Z depends on the classifier that we reimplemented. The test uses eight daily samples, the last of them a half day, so the smallest attainable two-sided p-value is 0.0078. With the Holm correction over all 36 comparisons of Table 5, 0 remain significant (smallest adjusted $p = 0.28$), including the comparison with B4. The cost differences we report are therefore descriptive. Their consistency across seeds, folds (Section 7.12), and noise variants (Section 7.9) carries more weight than any single test.

Fig. 3 shows that the flapping is harmless to remove. Among the 96 configuration changes of B4 (seed 0) in the test period, none saves more than 0.41% of the interval energy, 99% save less than 0.1%, and the median saving is below 0.01%. Such changes are expected when demand hovers near the boundary between two nearly equivalent configurations, and avoiding them costs essentially no energy.

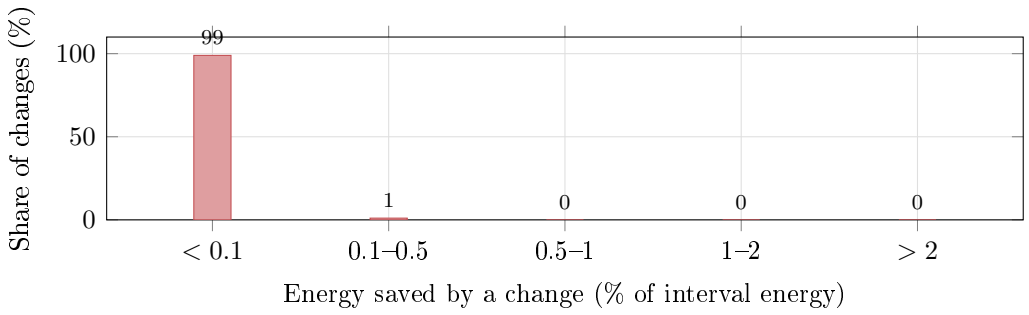


Figure 3: Energy gained by each configuration change of the snapshot classifier B4 (seed 0) on the Nobel-Germany test period (96 changes). 99% of the changes save less than 0.1% of the interval energy.

7.2. Other Scenarios and Where the Differences Come From

Table 5 summarizes all topologies and scenarios. Higher load and higher dynamic power change nothing on Abilene and GÉANT: the oracle remains a single configuration. On Nobel-Germany, higher load makes the optimal sequence non-constant (0.80 changes per day). B4 then makes 1.76 changes per day, B6 makes 1.07, and LiRA makes 0.80. LiRA's sequence equals the oracle's in every test interval and every seed (0 differing intervals), at a cost of 880.9 against 882.2 for B2, 887.0 for B6, and 890.7 for B4. The match has a simple cause. The expert is nearly constant, LiRA predicts the default configuration, and its safety layer, which sees the realized matrix, switches in exactly the intervals in which the default becomes infeasible, as the oracle does. The static rule reaches the same cost (880.9), and so does every ablation that keeps the safety layer (Section 7.4). Over 7.5 days of test data the difference to B6 is not significant ($p = 0.31$, LiRA cheaper on 50% of the days). Under stress, the only setting in which LiRA differs from the static rule, LiRA (898.6) is cheaper in total than B6 (899.4) and the static rule (902.0), but B2 (895.4) comes closer to the oracle (894.3). The difference to B6 is 0.09% of the cost, LiRA is cheaper on only 25% of the days, and the test gives $p = 0.64$.

Fig. 4 shows the busiest test week of the Nobel-Germany series. B4 changes configuration 88 times during the week, whereas LiRA makes only 2 changes. Fig. 5 plots total cost against configuration changes for the hysteresis and dwell-time sweeps and for the price sweep of LiRA and the oracle. LiRA and the oracle lie at the low-cost, low-change end of the plotted points,

Table 5: All Topologies and Scenarios: Configuration Changes per Day, Total Cost J, and Holm-Adjusted p-Values of the Wilcoxon Test on Per-Day Cost (B4, B6, and LiRA: Mean over Five Seeds)

		Changes/d				Total cost J					p_{Holm}	
Topology	Scenario	B4	B6	B2	LiRA	B4	B6	B2	LiRA	O*	vs B4	vs B6
Abilene	base	0.02	0.00	0.00	0.00	7227.7	7226.7	7230.4	7226.7	7226.7	0.539	1.000
Abilene	high load	0.00	0.00	0.02	0.00	7233.0	7233.0	7237.2	7233.0	7233.0	1.000	1.000
Abilene	high dynamic power	0.02	0.00	0.00	0.00	7248.7	7247.7	7258.7	7247.7	7247.7	0.539	1.000
Abilene	stress	0.00	0.00	0.02	0.00	7266.6	7266.6	7277.7	7266.6	7266.6	1.000	1.000
GÉANT	base	0.00	0.00	0.00	0.00	603.7	603.7	603.7	603.7	603.7	1.000	1.000
GÉANT	high load	0.00	0.00	0.00	0.00	610.4	610.4	610.4	610.4	610.4	1.000	1.000
GÉANT	high dynamic power	0.00	0.00	0.00	0.00	625.0	625.0	625.0	625.0	625.0	1.000	1.000
GÉANT	stress	0.00	0.00	0.00	0.00	644.8	644.8	644.8	644.8	644.8	1.000	1.000
Nobel-Germany	base	11.68	0.13	0.13	0.27	931.7	875.3	875.3	875.9	874.7	0.023	0.875
Nobel-Germany	high load	1.76	1.07	0.93	0.80	890.7	887.0	882.2	880.9	880.9	0.117	0.312
Nobel-Germany	high dynamic power	11.68	0.27	0.13	0.27	939.5	883.7	883.2	883.7	882.9	0.023	1.000
Nobel-Germany	stress	1.76	1.07	0.93	0.91	903.1	899.4	895.4	898.6	894.3	0.164	0.641

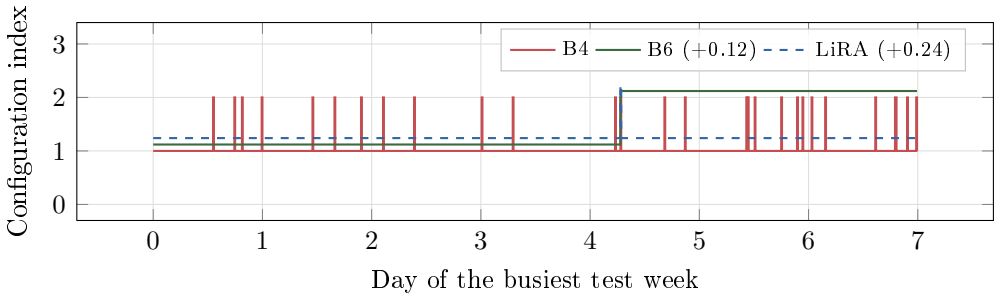


Figure 4: Installed configuration index over the busiest test week of the Nobel-Germany series. Offsets separate the lines. B4 switches repeatedly between two configurations, whereas B6 and LiRA hold a configuration.

and a tuned hysteresis threshold reaches nearly the same point.

7.3. Dependence on the Snapshot Classifier

The amount of flapping depends strongly on the classifier. On Nobel-Germany in the base scenario, B4 with XGBoost changes configuration 12.8 times per day, LightGBM 10.1, a random forest 0.5, and a multilayer perceptron 44.8 (Table S1 (Supplementary Material)). The random forest is nearly as stable as LiRA. Averaging over trees is a plausible reason, which we did not test. The perceptron flaps so much that its cost (1093.7) is far above that of LiRA (875.9). LiRA reaches the same cost with every backend, because its policy is constant in this scenario. A sequence-aware method is therefore

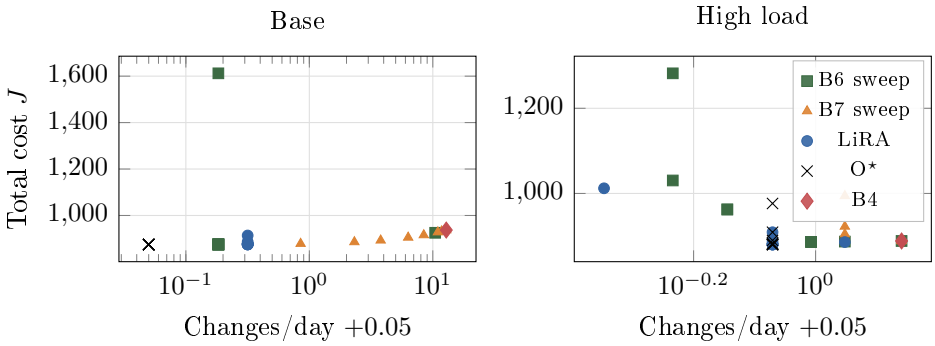


Figure 5: Total cost versus configuration changes on Nobel-Germany. Hysteresis thresholds and dwell times are swept for B6 and B7 and the reconfiguration price is swept for LiRA and the oracle. Each marker is one setting on the test split. The horizontal axis is logarithmic, and 0.05 is added to the change rate so that methods without changes can be shown.

most useful when the snapshot classifier is the kind that flaps, and it is least useful when a smoother classifier or a hysteresis guard is available.

7.4. Ablation, Library Size, and Threshold

Table S2 (Supplementary Material) removes one component of LiRA at a time on Nobel-Germany at the default price. It uses seeds 0 to 2, so the full policy's stress cost differs from the five-seed value in Table 5. In the high-load scenario the expert is nearly constant, so all variants that keep the safety layer reach the oracle cost. In the stress scenario the full policy costs 897.4 against 894.3 for the oracle. Removing the previous-configuration input raises the cost to 902.0, removing the cost-sensitive weights to 898.6, and removing DAgger to 899.0. Removing the lag features lowers it to 895.6. The differences are small and of mixed sign, and we did not test them for significance. Removing the safety layer lowers the cost to 889.9 because the policy then ignores the bound, and it causes 0.17% of the intervals to violate θ. Because J carries no penalty for violations, this cost is not comparable, and the layer is needed. On Abilene and GÉANT all variants coincide with the oracle because the expert is constant.

Table S3 (Supplementary Material) shows the effect of the library size on Nobel-Germany. Requested sizes above 51 yield the same library, because the candidate pool contains only 51 configurations. Every library size covers all test matrices. A larger library lowers the cost of B4 (from 939.3 at $K = 8$

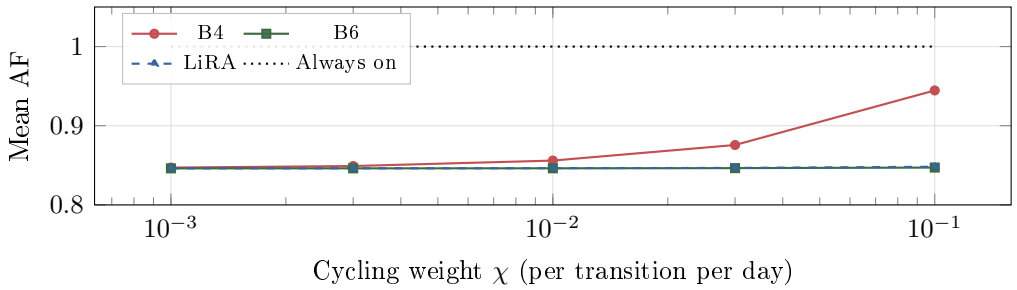


Figure 6: Mean acceleration factor of the decision sequences on Nobel-Germany for $\alpha = 0.5$. The factor of the always-on network is one.

to 935.7 at the largest size), but hysteresis and LiRA stay within 0.2% of the oracle cost at every size. Table S4 (Supplementary Material) varies the utilization threshold. At $\theta = 0.6$ B4 changes configuration 10.4 times per day and LiRA 0.40 times. For $\theta \geq 0.7$, each method gives the same result at every threshold on Nobel-Germany, because the same configurations are optimal and feasible. Table S4 (Supplementary Material) uses seed 0, so its B4 change rate differs from the five-seed mean of Table 4. On Abilene and GÉANT, the threshold has no effect between 0.6 and 0.9. LiRA never violates the bound when the library covers the traffic.

7.5. Wear and Lifetime

Fig. 6 shows the mean acceleration factor of the decision sequences on Nobel-Germany for $\alpha = 0.5$ and varying cycling weight χ (the sequences are fixed, and only the model changes). Every method stays below the always-on value of one for $\alpha = 0.5$ and all χ up to 0.1, but B4 degrades fastest: at $\chi = 0.1$ its mean factor is 0.94, against 0.85 for LiRA. At the default parameters (seed 0) the most-cycled link of B4 has a factor of 1.117, whereas the most-cycled link of LiRA has 1.0024 and those of B6, B7, and the oracle 1.000. When sleeping brings no benefit ($\alpha = 1$), every transition lowers lifetime, and every method that switches exceeds the always-on factor, B4 by the largest margin. Absolute lifetime figures depend on assumed parameters and should be read as relative comparisons.

7.6. Decision Time and Scalability

The library-based decisions are fast. The median decision time of B4 is 0.99 ms on Abilene, 0.95 ms on Nobel-Germany, and 1.60 ms on GÉANT.

LiRA needs less than 0.01 ms where its policy is constant and 1.571 ms where a tree model is evaluated. The safety check takes at most 0.36 ms even on a 65-node topology. Solving the integer program per traffic matrix (B1) takes a median of 537 ms on Nobel-Germany and 3.5 s on GÉANT, NSGA-II (B3) takes 28 s and 42 s, and the greedy heuristic is faster but gives away part of the savings (Table S5 (Supplementary Material)). On the subsample, the per-snapshot integer program reaches the same savings as the library snapshot optimum to within 0.007 percentage points. On larger SNDlib topologies the integer program takes 1.6 s for Germany50 and 11.4 s for TA2 (Table S6 (Supplementary Material)). These times fit five-minute intervals on the topologies we study, so speed alone does not require a learned controller here. The dynamic-programming oracle needs 3.6 s for all 48 108 Abilene intervals with $K = 32$, so prices can be explored offline.

7.7. What the Library Gives Up

Because the switching terms of (6) are non-negative, the sum over the test intervals of the optimal single-interval cost, with free link states and free routing over the k paths, is a lower bound on the cost J of any controller, with or without a library, including the optimum of the full multi-period mixed-integer program. We solve this single-interval integer program with HiGHS for all 21 203 test intervals of five settings (median 214 ms per interval, largest relative gap 9×10^{-7}) and use the solver's dual bound. Table 6 compares the bound with the library snapshot optimum, which picks the best library member in every interval and also ignores switching, with the oracle, and with LiRA. The library restriction itself costs at most 0.045% in any setting. The oracle lies within 0.55% of the bound, and on Nobel-Germany under high load and stress most of that gap is the cost of holding a configuration instead of re-optimizing every interval, which the bound ignores. A multi-period program without a library can therefore improve on the oracle by at most 0.55%.

7.8. Without the Wear Price

With the default prices, a sleeping link earns a wear credit about as large as its energy saving (Section 6), so J is well below the energy of the network. To check that this assumed credit does not drive the comparison, we repeat the main study with $\lambda_W = 0$: oracle labels, tuning of B6 and B7, training of B4 and LiRA, and the reported cost then contain energy and reconfiguration only. Table S7 (Supplementary Material) shows the result for all 12 settings.

Table 6: Lower Bound on the Cost of Any Controller (Sum of Per-Interval Optima), the Library Snapshot Optimum, the Oracle, and LiRA on the Test Split. Library Loss Is the Snapshot Optimum Above the Bound, O⋆ Gap the Oracle Above the Bound, Both Relative to the Oracle Cost

Setting	Bound	Lib. snapshot	O⋆	LiRA	Lib. loss (%)	O⋆ gap (%)
Abilene	7226.7	7226.7	7226.7	7226.7	0.000	0.000
GÉANT	603.7	603.7	603.7	603.7	0.011	0.011
Nobel, base	874.5	874.5	874.7	875.9	0.001	0.024
Nobel, high load	876.9	877.2	880.9	880.9	0.043	0.463
Nobel, stress	889.3	889.7	894.3	898.6	0.045	0.554

The oracle holds a single configuration in 9 of them. On Nobel-Germany B4 is again the most expensive online method (1.1% above the oracle in the base scenario), while B6, B2, the static rule, and LiRA lie within 0.12% of the oracle in every setting. LiRA is cheaper than every other online method in 0 settings and costs the same as the static rule in 10. Under high load it is slightly more expensive than the static rule (1570.2 against 1569.6, which equals the oracle). The ranking of the methods therefore does not depend on the wear model.

7.9. Noise Structure of the Semi-Synthetic Series

The only series with flapping is semi-synthetic, and its noise is independent across intervals. To see how the findings depend on this choice, we generate three further 30-day series with the same gravity model, weekly profile, split, and scaling. Two use log-normal noise with the same marginal scale but AR(1) correlation in time ($\rho = 0.5$ and $\rho = 0.9$), and one uses independent noise with half the scale. Each variant gets its own library, and every method is retrained (base scenario, five seeds). Table 7 shows that the amount of flapping depends strongly on the noise. With $\rho = 0.5$ the snapshot classifier changes configuration 57.8 times per day and costs 32.5% more than the oracle, because demands now stay near a decision boundary for several intervals and cross it repeatedly. With $\rho = 0.9$ the oracle itself switches (0.93 changes per day) and B4 is only 0.6% above it, and with half the noise B4 does not flap at all. In every variant, B6, B2, the static rule, and LiRA are at most 0.36% above the oracle, and LiRA costs the same as the static rule. The value of reconfiguration awareness thus ranges from nothing to a third of the cost, depending on the traffic, and in none of the variants does it require learning.

Table 7: Nobel-Germany Series with Different Noise (Base Scenario): Configuration Changes per Day, Total Cost J, and the Excess Cost of B4 over the Oracle. B4, B6, and LiRA: Mean over Five Seeds

	Changes/d				Total cost J						B4 above
Noise	B4	B6	LiRA	O*	B4	B6	B2	Static	LiRA	O*	O* (%)
i.i.d., $\sigma = 1.23$ (main)	11.68	0.13	0.27	0.00	931.7	875.3	875.3	875.9	875.9	874.7	6.5
AR(1) $\rho = 0.5$, $\sigma = 1.23$	57.81	0.00	0.00	0.00	1158.5	874.4	874.4	874.4	874.4	874.4	32.5
AR(1) $\rho = 0.9$, $\sigma = 1.23$	1.92	1.47	1.47	0.93	890.4	888.0	887.7	888.1	888.1	884.9	0.6
i.i.d., $\sigma = 0.62$	0.00	0.00	0.00	0.00	884.2	884.2	884.3	884.2	884.2	884.2	0.0

7.10. Traffic Drift

We inject three shifts into the test period of each topology: a ramp that raises all demands by up to 50% over ten days, a fourfold increase of three large demands, and a six-hour shift of the daily pattern. Table S8 (Supplementary Material) compares a LiRA policy trained once with drift-triggered retraining. On Abilene and GÉANT the shifts did not change which configuration this policy uses, because the library stays feasible. On Nobel-Germany the hotspot raised its configuration changes from 0.41 to 0.82 per day. The cost changed by at most 0.74% relative to the unshifted stream. The detectors were also not informative: without any shift, they raised 0.17, 0.00, and 0.93 false alarms per 1000 intervals on Abilene, GÉANT, and Nobel-Germany, and the Page–Hinkley threshold calibrated on validation data was at the maximum of the search grid (200) on the two real traces. We therefore cannot attribute detections to the injected shifts, and retraining did not lower the cost of LiRA. The injected shifts did not change the optimal configuration, so this experiment does not validate the detectors.

7.11. Decisions on the Previous Interval's Traffic

All results so far let the controller see D_t when it configures interval t (Section 6.3). Table 8 repeats the Nobel-Germany comparison when every online method decides with D_{t-1}: the features, the safety check, the hysteresis and dwell decisions, and the forecast of B2 all use the last completed interval, while costs and violations are measured on D_t (the same trained models, seeds 0 to 2). The decisions barely change, and costs move by at most 0.002% on Nobel-Germany and by less than 0.001% on Abilene and GÉANT. The utilization guarantee does not survive. On Nobel-Germany, B2, B4, and B6 violate θ in 0.05%, 0.19%, and 0.19% of the intervals in the base, high-load,

Table 8: Nobel-Germany: Total Cost J and Share of Intervals Violating θ When Every Online Method Knows D_t or Decides with D_{t-1}, Without and With a Safety Margin of 0.05 (B4, B6, and LiRA: Mean over Seeds 0 to 2)

	D_t known	D_{t-1}		D_{t-1}, margin 0.05	
Method	J	J	Viol. (%)	J	Viol. (%)
Nobel-Germany, base					
B2 rolling horizon	875.3	875.3	0.05	–	–
B4 snapshot ML	933.7	933.7	0.05	933.7	0.05
B6 hysteresis	875.3	875.3	0.05	875.3	0.05
Static + fallback	875.9	875.8	0.05	875.8	0.05
LiRA	875.9	875.8	0.05	875.8	0.05
Oracle O*	874.7	–	–	–	–
Nobel-Germany, high load					
B2 rolling horizon	882.2	882.2	0.19	–	–
B4 snapshot ML	891.2	891.2	0.19	897.1	0.19
B6 hysteresis	887.0	887.0	0.19	890.7	0.19
Static + fallback	880.9	880.9	0.14	886.2	0.14
LiRA	880.9	880.9	0.14	886.2	0.14
Oracle O*	880.9	–	–	–	–
Nobel-Germany, stress					
B2 rolling horizon	895.4	895.3	0.19	–	–
B4 snapshot ML	903.7	903.7	0.19	909.6	0.19
B6 hysteresis	899.4	899.4	0.19	903.9	0.19
Static + fallback	902.0	902.0	0.19	909.6	0.19
LiRA	897.4	897.4	0.17	901.4	0.17
Oracle O*	894.3	–	–	–	–

and stress scenarios, the same shares as the always-on network with shortest-path routing, and LiRA in 0.05%, 0.14%, and 0.17%. Every violated interval of the static rule is one in which the always-on network also violates θ, and in each of them the utilization of the installed configuration rises by 0.66 to 1.36 within one interval. Such peaks come from the independent per-interval noise of the series and cannot be predicted from D_{t-1}. A margin of 0.05 on θ therefore does not remove them, and it raises the cost (LiRA under high load: 886.2 against 880.9). On Abilene and GÉANT at most 0.008% of the intervals violate θ. LiRA's exact match with the oracle under high load thus depends on knowing the traffic of the interval it configures.

7.12. Rolling-Origin Folds

Table S9 (Supplementary Material) repeats the comparison for four rolling origins, retraining B4, B6, and LiRA on all data before each origin. In every fold B4 costs more than B6 and LiRA, and B6 and LiRA stay within 0.35% of the oracle. On Abilene and GÉANT, all methods coincide

in every fold. On Nobel-Germany, the mean cost over the folds is 371.5 for B4, 350.1 for B6, 357.9 for B7, 350.3 for LiRA, and 350.0 for the oracle.

7.13. Emulation of Configuration Changes

We emulate the most active day of the Nobel-Germany series in Mininet 2.3.0 with Open vSwitch 3.7.1 (17 switches, one host per switch, 26 links). Each configuration installs, for every origin–destination pair, the path that carries the largest share of its demand as OpenFlow rules with `ovs-ofctl`. A day is replayed with two seconds per interval. A configuration change wakes the links that the new configuration needs, waits one second of modeled wake-up time (Mininet does not model line-card physics), replaces the flow tables of the switches that change one after another, and then puts unused links to sleep. Eight UDP probe flows at 100 packets per second on the largest origin–destination pairs measure loss and one-way delay. B4 makes 18 configuration changes on this day, each toggling two links, whereas the sequences of B6, LiRA, and the oracle make none. Each change of B4 takes 1.11 s in total, which is the modeled wake-up time of 1 s plus 99 ms of measured rule installation and the time to put links to sleep. Table S10 (Supplementary Material) reports the probe loss. Each run sends 460 800 probe packets. B4 loses 3.7 packets per configuration change on average, all of them inside the windows around a change: 0.22% of the probes in those windows and 0.014% over the whole day. The always-on control and LiRA lose no probe packets, and LiRA keeps the same link-hours as B4, computed from the replayed configurations, which are 31% fewer than those of the always-on network. This emulation measures the cost of a single configuration change, not the energy of the network, and it ignores line-card boot time beyond the modeled delay.

8. Discussion

When does sequence awareness matter? In our data, it matters when the snapshot decision flaps. This happens with noisy demands whose fluctuations persist for a few intervals (Section 7.9) and with some classifiers (XGBoost, LightGBM, and the perceptron, but not the random forest), and it does not happen on smooth real traces. The wear model does not change this (Section 7.8). Where flapping occurs, holding a configuration removes nearly all changes without losing energy, and no learning is needed to do so: tuned hysteresis, a rolling-horizon optimizer with a persistence forecast, and a static configuration with a fallback all match LiRA.

What the oracle and LiRA add. The oracle makes the cost of any controller measurable on a trace. Together with the per-interval bound, it shows that the library loses at most 0.045% and that the real traces are nearly constant. LiRA's learned component changes the outcome in one setting, the stress scenario, where its total cost is 0.09% below that of hysteresis ($p = 0.64$) and above that of the rolling-horizon optimizer. Its exact match with the oracle under high load comes from the safety layer and from knowing the current traffic. With one-interval-old traffic LiRA keeps the oracle's cost but violates the utilization bound in 0.14% of the intervals.

Relation to learning-augmented algorithms. Our results match a lesson of online algorithms with predictions [45, 46]: a controller that follows a predictor blindly can pay far more than the optimum, and combining the predictor with a rule that holds its state recovers nearly all of the loss. In our setting the holding rule alone already suffices, because the oracle itself rarely switches.

Choosing the prices. The prices λ_R and λ_W are operator policy. The Pareto plots (Fig. 5) show how much cost a setting gives up, and because the oracle runs in seconds (Section 7.6), an operator can explore prices offline before deploying a policy.

Deployment. The safety layer and the all-on fallback are independent of the model, so a mispredicting model degrades to higher energy use and not to congestion, as long as the all-on configuration is feasible and the traffic estimate is accurate (Proposition 3). With one-interval-old estimates the layer no longer prevents violations on noisy traffic (Section 7.11), and a fixed margin does not help against single-interval peaks. A deployment therefore needs either a traffic forecast or capacity headroom sized to the peaks, which we did not evaluate. The fallback rate is also a monitoring signal for a stale library.

9. Limitations and Threats to Validity

- *Traffic.* The only series with strong flapping is semi-synthetic. We vary its noise structure (Section 7.9), but no real trace with such fluctuations was available to us. The evaluation on real traffic covers two research and education backbones, Abilene (2004) and GÉANT (2005), and both traces are old. The Nobel-Germany test split covers only 7.5 days, so tests on per-day costs have little power.

- *Power and wear.* The port power values and the wear parameters are assumed and not measured. The conclusion that the optimal configuration is stable depends on idle port power dominating dynamic power. A router with stronger load-proportional power, such as the high-dynamic-power scenario scaled further, may change this.

- *Baselines.* B4 is our reimplementation of the snapshot principle, not the published systems. B2 is restricted to the library. We did not solve the full multi-period mixed-integer program, but the per-interval bound limits what it can gain over the oracle (Section 7.7). B1, B3, and B5 were evaluated on subsamples. We did not train a deep reinforcement learning baseline.

- *Information.* The main results let every online method see the traffic of the interval it configures. With the previous interval's traffic, costs are unchanged but the safety layer admits violations on Nobel-Germany (Section 7.11). We did not evaluate traffic forecasts or estimation error from link loads.

- *Drift.* The injected shifts did not change the optimal configuration, and the detectors fired independently of them, so the drift study is inconclusive. Link failures and large library refreshes were not evaluated.

- *Emulation.* The emulation is small (17 switches, one day, compressed time), uses a modeled wake-up delay, single-path routing per demand, and no real line cards, and it reports a single run for LiRA.

- *Statistics.* Seeds vary only the learner. The traffic traces are fixed and the per-day tests use few days, so no comparison survives a correction over all settings.

- *Learned component.* In 30 of 60 LiRA runs the trained policy is constant, in 56 of them the selected policy predates DAgger, and the static rule equals LiRA in 11 of 12 settings. A learned policy that does more than hold a configuration is therefore evaluated only in the Nobel-Germany stress scenario (898.6 against 894.3 for the oracle, five seeds). On GÉANT the classifier is not constant as a model, but its output is constant on the test split.

10. Conclusion

We formulated energy-aware SDN configuration as a sequential problem, solved it exactly over a configuration library, bounded what the library gives up, and used the solution to train a history-conditioned policy with a safety layer. The empirical result is a qualified one. On two real traces, a snapshot classifier is already within 0.014% of the oracle, and the oracle and LiRA hold a single configuration. On noisy semi-synthetic series, a snapshot classifier flaps and costs up to 32.5% more than the oracle, and holding a configuration removes nearly all of that excess at no loss of energy. This holds without the wear price and for every noise variant we tried. Tuned hysteresis, a rolling-horizon optimizer, and a static configuration with a fallback do the same, and LiRA's learned component changes the result in one scenario only, by an amount that is not significant. The safety layer guarantees the utilization bound only when the traffic of the configured interval is known. We release the oracle, the baselines, and all scripts (`https://github.com/adeliusa486/-lira-sdn-energy`) so that other traces and power models can be tested. The extensions we see are traffic with realistic temporal structure, measured router power, controllers that forecast traffic, and hardware with sleep-capable line cards.

Data availability

The traffic matrices are public: Abilene, GÉANT, and the one-day Nobel-Germany series come from SNDlib [60, 59, 54], and the 30-day Nobel-Germany series is generated by a script in the released code. The code, configuration libraries, per-experiment CSV files, and scripts that produce every table and figure of this article are available at `https://github.com/adeliusa486/-lira-sdn-energy`.

CRediT authorship contribution statement

Adeel Ahmad: Conceptualization, Methodology, Software, Investigation, Data curation, Visualization, Writing – original draft. **Toqeer Ali Syed:** Supervision, Validation, Writing – review & editing. **Ali Akarma:** Formal analysis, Validation, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] M. Gupta, S. Singh, Greening of the internet, in: Proc. ACM SIGCOMM, 2003, pp. 19–26. doi:10.1145/863955.863959.

[2] A. P. Bianzino, C. Chaudet, D. Rossi, J.-L. Rougier, A survey of green networking research, IEEE Communications Surveys & Tutorials 14 (1) (2012) 3–20. doi:10.1109/SURV.2011.113010.00106.

[3] R. Bolla, R. Bruschi, F. Davoli, F. Cucchietti, Energy efficiency in the future internet: A survey of existing approaches and trends in energy-aware fixed network infrastructures, IEEE Communications Surveys & Tutorials 13 (2) (2011) 223–244. doi:10.1109/SURV.2011.071410.00073.

[4] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: Enabling innovation in campus networks, ACM SIGCOMM Computer Communication Review 38 (2) (2008) 69–74. doi:10.1145/1355734.1355746.

[5] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, Proceedings of the IEEE 103 (1) (2015) 14–76. doi:10.1109/JPROC.2014.2371999.

[6] L. Chiaraviglio, M. Mellia, F. Neri, Minimizing ISP network energy cost: Formulation and solutions, IEEE/ACM Transactions on Networking 20 (2) (2012) 463–476. doi:10.1109/TNET.2011.2161487.

[7] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, On optimizing energy efficiency in SDN networks through ML-driven configuration, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2025. doi:10.1109/NOMS57970.2025.11073657.

[8] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, AI-driven scalable framework for energy-efficient SDN configurations, IEEE Transactions on Green Communications and Networking (2026). doi:10.1109/TGCN.2026.3724271.

[9] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, M. Yu, Teal: Learning-accelerated optimization of WAN traffic engineering, in: Proc. ACM SIGCOMM, 2023. doi:10.1145/3603269.3604857.

[10] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, A. Tamar, DOTE: Rethinking (predictive) WAN traffic engineering, in: Proc. USENIX NSDI, 2023.
URL https://www.usenix.org/conference/nsdi23/presentation/perry

[11] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, P. Bailis, Contracting wide-area network topologies to solve flow problems quickly, in: Proc. USENIX NSDI, 2021.
URL https://www.usenix.org/conference/nsdi21/presentation/abuzaid

[12] P. Almasan, S. Xiao, X. Cheng, X. Shi, P. Barlet-Ros, A. Cabellos-Aparicio, ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning, Computer Networks 214 (2022) 109166. doi:10.1016/j.comnet.2022.109166.

[13] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. doi:10.1016/j.ejor.2020.07.063.

[14] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, D. Walker, Abstractions for network update, in: Proc. ACM SIGCOMM, 2012, pp. 323–334. doi:10.1145/2342356.2342427.

[15] X. Jin, H. H. Liu, R. Gandhi, S. Kandula, R. Mahajan, M. Zhang, J. Rexford, R. Wattenhofer, Dynamic scheduling of network updates, in: Proc. ACM SIGCOMM, 2014, pp. 539–550. doi:10.1145/2619239.2626307.

[16] L. Chiaraviglio, P. Wiatr, P. Monti, J. Chen, J. Lorincz, F. Idzikowski, M. Listanti, L. Wosinska, Is green networking beneficial in terms of device lifetime?, IEEE Communications Magazine 53 (5) (2015) 232–240. doi:10.1109/MCOM.2015.7105670.

[17] L. Chiaraviglio, et al., Impact of energy-efficient techniques on a device lifetime, in: Proc. IEEE Online Conference on Green Communications (OnlineGreenComm), 2014. doi:10.1109/OnlineGreenCom.2014.7114426.

[18] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proc. AISTATS, 2011, pp. 627–635.
URL https://proceedings.mlr.press/v15/ross11a.html

[19] M. Zhang, C. Yi, B. Liu, B. Zhang, GreenTE: Power-aware traffic engineering, in: Proc. IEEE Int. Conf. Network Protocols (ICNP), 2010, pp. 21–30. doi:10.1109/ICNP.2010.5762751.

[20] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, N. McKeown, ElasticTree: Saving energy in data center networks, in: Proc. USENIX NSDI, 2010.
URL https://www.usenix.org/conference/nsdi10-0/elastictree-saving-energy-data-center-networks

[21] N. Vasić, D. Kostić, Energy-aware traffic engineering, in: Proc. ACM e-Energy, 2010. doi:10.1145/1791314.1791341.

[22] P. Mahadevan, P. Sharma, S. Banerjee, P. Ranganathan, A power benchmarking framework for network devices, in: Proc. IFIP Networking, Vol. 5550 of LNCS, 2009, pp. 795–808. doi:10.1007/978-3-642-01399-7_62.

[23] J. C. C. Restrepo, C. G. Gruber, C. M. Machuca, Energy profile aware routing, in: Proc. IEEE ICC Workshops, 2009. doi:10.1109/ICCW.2009.5208041.

[24] A. Vishwanath, K. Hinton, R. W. A. Ayre, R. S. Tucker, Modeling energy consumption in high-capacity routers and switches, IEEE Journal on Selected Areas in Communications 32 (8) (2014) 1524–1532. doi:10.1109/JSAC.2014.2335312.

[25] R. Maaloul, R. Taktak, L. Chaari, B. Cousin, Energy-aware routing in carrier-grade Ethernet using SDN approach, IEEE Transactions on Green Communications and Networking 2 (3) (2018) 844–858. doi:10.1109/TGCN.2018.2832658.

[26] B. G. Assefa, Ö. Özkasap, A survey of energy efficiency in SDN: Software-based methods and optimization models, Journal of Network and Computer Applications 137 (2019) 127–143. doi:10.1016/j.jnca.2019.04.001.

[27] B. G. Assefa, Ö. Özkasap, MER-SDN: Machine learning framework for traffic aware energy efficient routing in SDN, in: Proc. IEEE DASC/PiCom/DataCom/CyberSciTech, 2018. doi:10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.000-1.

[28] P. Fröhlich, E. Gelenbe, M. P. Nowak, Reinforcement learning and energy-aware routing, in: Proc. ACM FlexNets Workshop, 2021. doi:10.1145/3472735.3473390.

[29] X. Zheng, W. Huang, S. Wang, J. Zhang, H. Zhang, Research on energy-saving routing technology based on deep reinforcement learning, Electronics 11 (13) (2022) 2035. doi:10.3390/electronics11132035.

[30] Q. Wu, X. Chen, Z. Zhou, L. Chen, J. Zhang, Deep reinforcement learning with spatio-temporal traffic forecasting for data-driven base station sleep control, IEEE/ACM Transactions on Networking 29 (2) (2021) 935–948. doi:10.1109/TNET.2021.3053771.

[31] Y. Wang, Y. Li, T. Wang, G. Liu, Towards an energy-efficient data center network based on deep reinforcement learning, Computer Networks 210 (2022) 108939. doi:10.1016/j.comnet.2022.108939.

[32] X. Pei, P. Sun, Y. Hu, D. Li, B. Chen, L. Tian, Enabling efficient routing for traffic engineering in SDN with deep reinforcement learning, Computer Networks 241 (2024) 110220. doi:10.1016/j.comnet.2024.110220.

[33] I. Maity, S. Misra, C. Mandal, ETHoS: Energy-aware traffic engineering for sustainable hybrid SDN, IEEE Transactions on Sustainable Computing 7 (4) (2022) 875–886. doi:10.1109/TSUSC.2022.3164571.

[34] L. Röllin, R. Jacob, L. Vanbever, A sleep study for ISP networks: Evaluating link sleeping on real world data, ACM SIGEnergy Energy Informatics Review 4 (2024) 51–58. doi:10.1145/3727200.3727209.

[35] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, M. Zaharia, Solving large-scale granular resource allocation problems efficiently with POP, in: Proc. ACM SOSP, 2021, pp. 521–537. doi:10.1145/3477132.3483588.

[36] A. Valadarsky, M. Schapira, D. Shahaf, A. Tamar, Learning to route, in: Proc. ACM HotNets, 2017, pp. 185–191. doi:10.1145/3152434.3152441.

[37] G. Bernárdez, J. Suárez-Varela, A. López, X. Shi, S. Xiao, X. Cheng, P. Barlet-Ros, A. Cabellos-Aparicio, MAGNNETO: A graph neural network-based multi-agent system for traffic engineering, IEEE Transactions on Cognitive Communications and Networking 9 (2) (2023) 494–506. doi:10.1109/TCCN.2023.3235719.

[38] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, A. Cabellos-Aparicio, RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN, IEEE Journal on Selected Areas in Communications 38 (10) (2020) 2260–2270. doi:10.1109/JSAC.2020.3000405.

[39] M. Ye, J. Zhang, Z. Guo, H. J. Chao, FlexDATE: Flexible and disturbance-aware traffic engineering with reinforcement learning in software-defined networks, IEEE/ACM Transactions on Networking 31 (4) (2023) 1433–1448. doi:10.1109/TNET.2022.3217083.

[40] E. Bonetto, L. Chiaraviglio, F. Idzikowski, E. Le Rouzic, Algorithms for the multi-period power-aware logical topology design with reconfiguration costs, Journal of Optical Communications and Networking 5 (5) (2013) 394–410. doi:10.1364/JOCN.5.000394.

[41] P. Wiatr, et al., Lifetime-aware provisioning in green optical backbone networks, in: Proc. Optical Fiber Communication Conference (OFC), 2016. doi:10.1364/OFC.2016.M3K.1.

[42] A. Borodin, N. Linial, M. E. Saks, An optimal on-line algorithm for metrical task system, Journal of the ACM 39 (4) (1992) 745–763. doi:10.1145/146585.146588.

[43] N. Chen, G. Goel, A. Wierman, Smoothed online convex optimization in high dimensions via online balanced descent (2018). arXiv:1803.10366. URL https://arxiv.org/abs/1803.10366

[44] M. Lin, A. Wierman, L. L. H. Andrew, E. Thereska, Dynamic right-sizing for power-proportional data centers, IEEE/ACM Transactions on Networking 21 (5) (2013) 1378–1391. doi:10.1109/TNET.2012.2226216.

[45] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, Journal of the ACM 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.

[46] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, ACM Transactions on Algorithms 19 (2) (2023) 19:1–19:34. doi:10.1145/3582689.

[47] N. Christianson, J. Shen, A. Wierman, Optimal robustness-consistency tradeoffs for learning-augmented metrical task systems, in: Proc. 26th Int. Conf. Artificial Intelligence and Statistics (AISTATS), Vol. 206 of PMLR, 2023, pp. 9377–9399.

[48] D. Rutten, N. Christianson, D. Mukherjee, A. Wierman, Smoothed online optimization with unreliable predictions, Proc. ACM Meas. Anal. Comput. Syst. 7 (1) (2023) 12:1–12:36. doi:10.1145/3579442.

[49] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Computing Surveys 46 (4) (2014) 44:1–44:37. doi:10.1145/2523813.

[50] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, IEEE Transactions on Knowledge and Data

Engineering 31 (12) (2019) 2346–2363. doi:10.1109/TKDE.2018.2876857.

[51] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: Proc. SIAM Int. Conf. Data Mining (SDM), 2007, pp. 443–448. doi:10.1137/1.9781611972771.42.

[52] E. S. Page, Continuous inspection schemes, Biometrika 41 (1/2) (1954) 100–115. doi:10.1093/biomet/41.1-2.100.

[53] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, L. Cavallaro, INSOMNIA: Towards concept-drift robustness in network intrusion detection, in: Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2021, pp. 111–122. doi:10.1145/3474369.3486864.

[54] Y. Zhang, M. Roughan, N. Duffield, A. Greenberg, Fast accurate computation of large-scale IP traffic matrices from link loads, in: Proc. ACM SIGMETRICS, 2003, pp. 206–217. doi:10.1145/781027.781053.

[55] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

[56] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proc. ACM SIGKDD, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[57] G. Ke, et al., LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.
URL https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree

[58] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[59] S. Uhlig, B. Quoitin, J. Lepropre, S. Balon, Providing public intradomain traffic matrices to the research community, ACM SIGCOMM Computer Communication Review 36 (1) (2006) 83–86. doi:10.1145/1111322.1111341.

[60] S. Orlowski, R. Wessäly, M. Pióro, A. Tomaszewski, SNDlib 1.0—survivable network design library, Networks 55 (3) (2010) 276–286. doi:10.1002/net.20371.

[61] Q. Huangfu, J. A. J. Hall, Parallelizing the dual revised simplex method, Mathematical Programming Computation 10 (1) (2018) 119–142. doi:10.1007/s12532-017-0130-5.